

Additional II

Increasing Array II

```
#include

using namespace std;
typedef long long ll;

int N, x;
priority_queue Q;

int main(){
    scanf("%d", &N);
    ll ans = 0;
    for(int i = 0; i < N; i++){
        scanf("%d", &x);
        Q.push(x);
        ans += Q.top() - x;
        Q.pop();
        Q.push(x);
    }
    printf("%lld\n", ans);
}
```

Food Division

```
#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;

int N, A[maxN];

int main(){
    scanf("%d", &N);
    for(int i = 1; i <= N; i++) scanf("%d", &A[i]);
    for(int i = 1, b; i <= N; i++){
        scanf("%d", &b);
        A[i] += (A[i-1] - b);
    }
    sort(A+1, A+N+1);

    ll ans = 0;
    for(int i = 1; i <= N; i++)
        ans += abs(A[i] - A[(N+1)/2]);
    printf("%lld\n", ans);
}
```

Swap Round Sorting

```
#include

using namespace std;
typedef pair pii;
const int maxN = 2e5+1;

bool vis[maxN];
int N, a[maxN];
vector cycle;
vector ans;

bool zero_rounds(){
    for(int i = 1; i <= N; i++){
        if(a[i] != i)
            return false;

        printf("0\n");
        return true;
    }

    bool one_round(bool second = false){
        for(int i = 1; i <= N; i++){
            if(a[i] != i){
                if(i != a[a[i]]) { ans.clear(); return false; }
                else if(i < a[i]) ans.push_back({i, a[i]});
            }
        }

        if(!second) printf("1\n");
        printf("%d\n", (int) ans.size());
        for(pii p : ans){
            printf("%d %d\n", p.first, p.second);
            swap(a[p.first], a[p.second]);
        }
        return true;
    }

    void dfs(int u){
        vis[u] = true;
        cycle.push_back(u);
        if(!vis[a[u]]) dfs(a[u]);
    }

    int main(){
        scanf("%d", &N);
        for(int i = 1; i <= N; i++){
            scanf("%d", &a[i]);
        }

        if(!zero_rounds() && !one_round()){
            printf("2\n");

            for(int i = 1; i <= N; i++){
                if(!vis[i]){
                    dfs(i);
                    int K = (int) cycle.size();
                    for(int u = 0; u < K/2; u++){
                        ans.push_back({cycle[u], cycle[K-u-1]});
                        swap(a[cycle[u]], a[cycle[K-u-1]]);
                    }
                    cycle.clear();
                }
            }

            printf("%d\n", (int) ans.size());
            for(pii p : ans)
                printf("%d %d\n", p.first, p.second);
        }
    }
}
```

```

        ans.clear();
        one_round(true);
    }
}

```

School Excursion

```

#include

using namespace std;
const int maxN = 1e5+1;

int N, M, a, b, ds[maxN];
multiset sizes;
bitset dp;

int find(int u){
    if(ds[u] < 0) return u;
    ds[u] = find(ds[u]);
    return ds[u];
}

bool merge(int u, int v){
    u = find(u); v = find(v);
    if(u == v) return false;
    if(ds[u] < ds[v]) swap(u, v);
    ds[v] += ds[u];
    ds[u] = v;
    return true;
}

int main(){
    scanf("%d %d", &N, &M);
    fill(ds+1, ds+N+1, -1);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        merge(a, b);
    }

    for(int i = 1; i <= N; i++)
        if(find(i) == i)
            sizes.insert(-ds[i]);

    dp[0] = 1;
    for(int sz : sizes)
        dp |= (dp <

```

Coin Grid

```

#include

```

```

using namespace std;
typedef pair pii;
const int maxN = 105;

char S[maxN];
bool used[maxN], inZ[2][maxN];
int N, cnt, mt[maxN];
vector G[maxN];

bool kuhns(int u){
    if(used[u]) return false;

    used[u] = true;
    for(int v : G[u]){
        if(!mt[v] || kuhns(mt[v])){
            mt[v] = u;
            return true;
        }
    }

    return false;
}

void dfs(int t, int u){
    inZ[t][u] = true;
    if(t == 0 && !inZ[1][mt[u]]){
        dfs(1, mt[u]);
    } else if(t == 1){
        for(int v : G[u])
            if(!inZ[0][v])
                dfs(0, v);
    }
}

int main(){
    scanf("%d", &N);
    for(int i = 1; i <= N; i++){
        scanf(" %s", S);
        for(int j = 1; j <= N; j++){
            if(S[j-1] == 'o')
                G[j].push_back(i);
        }

        for(int i = 1; i <= N; i++){
            kuhns(i);
            fill(used+1, used+N+1, false);
        }

        cnt = 0;
        for(int i = 1; i <= N; i++){
            if(mt[i]){
                cnt++;
                used[mt[i]] = true;
            }
        }

        for(int i = 1; i <= N; i++)
            if(!used[i])
                dfs(1, i);

        printf("%d\n", cnt);
        for(int i = 1; i <= N; i++)
            if(inZ[0][i])
                printf("1 %d\n", i);
        for(int i = 1; i <= N; i++)
            if(!inZ[1][i])
                printf("2 %d\n", i);
    }
}

```

Programmers and Artists

```
#include

using namespace std;
typedef long long ll;
typedef pair pii;
const int maxN = 2e5+5;

int N, A, B;
pii cand[maxN];

multiset initial_artist_set(){
    vector all;
    for(int i = A; i < N; i++){
        all.push_back(cand[i].second);
    }
    sort(all.begin(), all.end());

    reverse(all.begin(), all.end());
    multiset best;
    for(int i = 0; i < B; i++){
        best.insert(all[i]);
    }
    return best;
}

int main(){
    scanf("%d %d %d", &A, &B, &N);
    for(int i = 0; i < N; i++){
        scanf("%d %d", &x, &y);
        cand[i] = {x, y};
    }
    sort(cand, cand+N, [](const pii a, const pii b){
        return a.first == b.first ? a.second < b.second : a.first > b.first;
    });

    ll pref = 0;
    priority_queue prefix_deltas;
    for(int i = 0; i < A; i++){
        pref += cand[i].first;
        prefix_deltas.push(cand[i].second - cand[i].first);
    }

    multiset remaining_art = initial_artist_set();
    ll suf = accumulate(remaining_art.begin(), remaining_art.end(), 0LL);

    ll ans = pref + suf;
    for(int i = A; i < A+B; i++){
        const int x = cand[i].first;
        const int y = cand[i].second;
        pref += x;

        prefix_deltas.push(y - x);
        pref += prefix_deltas.top();
        prefix_deltas.pop();

        auto worst_artist = remaining_art.lower_bound(y);
        int art_value = *worst_artist;
        remaining_art.erase(worst_artist);
        suf -= art_value;

        ans = max(ans, pref + suf);
    }
}
```

```
    printf("%lld\n", ans);  
}
```

Coin Arrangement

```
#include  
  
using namespace std;  
typedef long long ll;  
const int maxN = 1e5;  
  
int N, a[2][maxN];  
  
int main(){  
    scanf("%d", &N);  
    for(int r = 0; r < 2; r++){  
        for(int i = 0; i < N; i++){  
            scanf("%d", &a[r][i]);  
            a[r][i]--;  
        }  
    }  
  
    ll ans = 0;  
    int top = 0, bot = 0;  
    for(int i = 0; i < N; i++){  
        top += a[0][i];  
        bot += a[1][i];  
  
        if((top < 0 && bot > 0) || (top > 0 && bot < 0)){  
            if(abs(top) < abs(bot)){  
                ans += abs(top);  
                bot += top;  
                top = 0;  
            } else {  
                ans += abs(bot);  
                top += bot;  
                bot = 0;  
            }  
        }  
  
        ans += abs(top + bot);  
    }  
    printf("%lld\n", ans);  
}
```

Grid Puzzle I

```
#include  
  
using namespace std;  
typedef pair pii;  
const int maxN = 105, maxM = 2605;
```

```

const int INF = 0x3f3f3f3f;

int N, rowTot, colTot, edgeID, p[maxN], cap[maxN][maxN];
bool vis[maxM];
vector path, F[maxN];
vector G[maxN];

int bfs(int s = 0, int t = 2*N+1){
    fill(p, p+2*N+2, -1);
    p[s] = -2;

    queue Q;
    Q.push({s, INF});
    while(!Q.empty()){
        int u = Q.front().first;
        int f = Q.front().second;
        Q.pop();

        for(int v : F[u]){
            if(p[v] == -1 && cap[u][v]){
                p[v] = u;
                int aug = min(f, cap[u][v]);
                if(v == t) return aug;
                Q.push({v, aug});
            }
        }
    }

    return 0;
}

void dfs(int u = 0){
    path.push_back(u);
    if(u == N) return;
    for(pii e : G[u]){
        int v = e.first;
        int id = e.second;
        if(cap[u][v] == 0 && !vis[id]){
            vis[id] = true;
            dfs(v);
            return;
        }
    }
}

int maxflow(int s = 0, int t = 2*N+1){
    int flow = 0, aug = 0;
    while(aug = bfs()){
        flow += aug;
        int u = t;
        while(u != s){
            int v = p[u];
            cap[v][u] -= aug;
            cap[u][v] += aug;
            u = v;
        }
    }
    return flow;
}

int main(){
    scanf("%d", &N);
    for(int i = 1, c; i <= N; i++){
        scanf("%d", &c);
        G[0].push_back({i, ++edgeID});
        F[0].push_back(i);
        F[i].push_back(0);
        cap[0][i] += c;
        rowTot += c;
    }
}

```

```

    for(int i = N+1, c; i <= 2*N; i++){
        scanf("%d", &c);
        G[i].push_back({2*N+1, ++edgeID});
        F[i].push_back(2*N+1);
        F[2*N+1].push_back(i);
        cap[i][2*N+1] += c;
        colTot += c;
    }
    for(int i = 1; i <= N; i++){
        for(int j = N+1; j <= 2*N; j++){
            G[i].push_back({j, ++edgeID});
            F[i].push_back(j);
            F[j].push_back(i);
            cap[i][j]++;
        }
    }

    int K = maxflow();
    if(rowTot != colTot || K != rowTot){
        printf("-1\n");
        return 0;
    }

    for(int i = 1; i <= N; i++){
        for(int j = 1; j <= N; j++){
            printf("%c", (cap[i][j+N] ? '.' : 'X'));
            printf("\n");
        }
    }
}

```

Grid Puzzle II

```

#include

using namespace std;
const int maxN = 105, maxM = 2605;
const int INF = 0x3f3f3f3f;

int N, K, rowtot, coltot, edgecnt, p[maxN], d[maxN];
bool inq[maxN], vis[maxM];
vector path, G[maxN];

struct Edge {
    int u, v, r, c;
} edges[maxM], redges[maxM];

void read_input(){
    scanf("%d", &N);
    for(int v = 1, cap; v <= N; v++){
        scanf("%d", &cap);
        rowtot += cap;
        edgecnt++;
        G[0].push_back(edgecnt);
        G[v].push_back(-edgecnt);
        edges[edgecnt] = {0, v, cap, 0};
        redges[edgecnt] = {v, 0, 0, 0};
    }
    for(int u = N+1, cap; u <= 2*N; u++){
        scanf("%d", &cap);
        coltot += cap;
        edgecnt++;
    }
}

```

```

G[u].push_back(edgecnt);
G[2*N+1].push_back(-edgecnt);
edges[edgecnt] = {u, 2*N+1, cap, 0};
redges[edgecnt] = {2*N+1, u, 0, 0};
}
for(int u = 1; u <= N; u++){
    for(int v = N+1, cost; v <= 2*N; v++){
        scanf("%d", &cost);
        edgecnt++;
        G[u].push_back(edgecnt);
        G[v].push_back(-edgecnt);
        edges[edgecnt] = {u, v, 1, -cost};
        redges[edgecnt] = {v, u, 0, cost};
    }
}
}

void bellman_ford(int s = 0){
    fill(inq, inq+2*N+2, false);
    fill(d, d+2*N+2, INF);
    fill(p, p+2*N+2, 0);

    queue Q;
    Q.push(s);
    d[s] = 0;
    inq[s] = true;
    while(!Q.empty()){
        int u = Q.front(); Q.pop();
        inq[u] = false;

        for(int i : G[u]){
            Edge e = (i < 0 ? redges[-i] : edges[i]);
            if(e.r > 0 && d[e.v] > d[u] + e.c){
                d[e.v] = d[u] + e.c;
                p[e.v] = i;
                if(!inq[e.v]){
                    inq[e.v] = true;
                    Q.push(e.v);
                }
            }
        }
    }
}

int minimum_cost_flow(int s = 0, int t = 2*N+1){
    int flow = 0, cost = 0;
    while(flow < K){
        bellman_ford(s);
        if(d[t] == INF) break;

        int aug = K-flow;
        int u = t;
        while(u != s){
            Edge e = (p[u] < 0 ? redges[-p[u]] : edges[p[u]]);
            aug = min(aug, e.r);
            u = e.u;
        }

        flow += aug;
        cost += aug * d[t];
        u = t;
        while(u != s){
            if(p[u] < 0){
                redges[-p[u]].r -= aug;
                edges[-p[u]].r += aug;
            } else {
                redges[p[u]].r += aug;
                edges[p[u]].r -= aug;
            }
            u = (p[u] < 0 ? redges[-p[u]].u : edges[p[u]].u);
        }
    }
}

```

```

    }
}
return (flow < K ? INF : cost);
}

int main(){
    read_input();

    K = rowtot;
    int max_reward = -minimum_cost_flow();
    if(rowtot != coltot || max_reward == -INF){
        printf("-1\n");
        return 0;
    }

    printf("%d\n", max_reward);
    for(Edge e : edges){
        if(e.u != 0 && e.v != 2*N+1){
            printf("%c", (e.r == 0 ? 'X' : '.'));
            if(e.v == 2*N) printf("\n");
        }
    }
}

```

Bit Substrings

```

#include

using namespace std;
typedef long long ll;
typedef double ld;
typedef complex cd;
const int maxN = 2e5+5;
const ld PI = acos(-1);

ll ans[maxN];
char S[maxN];

void fft(vector &a, bool inv){
    int N = (int) a.size();

    for(int i = 1, j = 0; i < N; i++){
        int bit = N>>1;
        for(; j&bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if(i < j)
            swap(a[i], a[j]);
    }

    for(int len = 2; len <= N; len <= 1){
        ld theta = 2*PI / len * (inv ? -1 : 1);
        cd wlen(cos(theta), sin(theta));
        for(int i = 0; i < N; i += len){
            cd w(1);
            for(int j = 0; j < len / 2; j++){
                cd u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }
    }
}

```

```

    }
}

if(inv)
    for(cd &z : a)
        z /= N;
}

int next_two_pow(int x){
    return 1<<(32 - __builtin_clz(x));
}

void solve(int l, int r){
    if(l+1 == r){
        ans[(int) (S[l] - '0')]++;
        return;
    }

    int m = (l+r)/2;
    int sz = next_two_pow(r-l);
    vector A(sz), B(sz);
    for(int i = m-1, cnt = 0; i >= l; i--){
        if(S[i] == '1') cnt++;
        A[cnt] += 1;
    }
    for(int i = m, cnt = 0; i < r; i++){
        if(S[i] == '1') cnt++;
        B[cnt] += 1;
    }

    fft(A, false);
    fft(B, false);
    for(int i = 0; i < sz; i++)
        A[i] *= B[i];
    fft(A, true);

    for(int i = 0; i < sz && i < maxN; i++)
        ans[i] += llround(A[i].real());

    solve(l, m);
    solve(m, r);
}

int main(){
    scanf("%s", S);
    int N = (int) strlen(S);
    solve(0, N);
    for(int i = 0; i <= N; i++)
        printf("%lld%c", ans[i], (" \n")[i==N]);
}

```

Reversal Sorting

```

#include

using namespace std;
const int maxN = 2e5+5;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
uniform_int_distribution dist(1, (int) 2e9);

```

```

struct Node {
    bool rev;
    int value, prior, sz;
    Node *p, *l, *r;

    Node(){}
    Node(int val, int pri){
        prior = pri;
        value = val;
        sz = 1;
        p = l = r = nullptr;
    }
};

int N, X[maxN];
Node *root, *ndptr[maxN];

int sz(Node *t){    return t ? t->sz : 0;    }
int val(Node *t){   return t ? t->value : -1; }

void flip(Node *t){
    if(!t) return;
    t->rev ^= true;
}

void pull(Node *t){
    if(!t) return;
    if(t->l) t->l->p = t;
    if(t->r) t->r->p = t;
    t->sz = sz(t->l) + sz(t->r) + 1;
}

void push(Node *t){
    if(!t) return;
    if(t->rev){
        swap(t->l, t->r);
        flip(t->l); flip(t->r);
        t->rev = false;
    }
}

Node* merge(Node *x, Node *y){
    if(!x || !y) return x ? x : y;

    push(x); push(y);
    if(x->prior < y->prior){
        x->r = merge(x->r, y);
        pull(x);
        return x;
    } else {
        y->l = merge(x, y->l);
        pull(y);
        return y;
    }
}

pair split(Node *x, int k){
    if(!x) return {nullptr, nullptr};

    pair y = {nullptr, nullptr};
    push(x);
    if(k <= sz(x->l)){
        y = split(x->l, k);
        x->l = y.second;
        pull(x);
        y.second = x;
    } else {
        y = split(x->r, k-sz(x->l)-1);
        x->r = y.first;
    }
}

```

```

        pull(x);
        y.first = x;
    }
    return y;
}

void heapify(Node *t){
    if(!t) return;

    Node *mx = t;
    if(t->l && t->l->prior > mx->prior) mx = t->l;
    if(t->r && t->r->prior > mx->prior) mx = t->r;
    if(mx != t){
        swap(t->prior, mx->prior);
        heapify(mx);
    }
}

Node* build(int x, int k){
    if(k == 0) return nullptr;

    int mid = k/2;
    Node *t = new Node(X[x+mid], dist(rng));
    ndptr[X[x+mid]] = t;
    t->l = build(x, mid);
    t->r = build(x+mid+1, k-mid-1);
    heapify(t);
    pull(t);
    return t;
}

void reverse(int x, int k){
    pair y, z;
    y = split(root, x-1);
    z = split(y.second, k);
    flip(z.first);
    y.second = merge(z.first, z.second);
    root = merge(y.first, y.second);
    root->p = nullptr;
}

int orderOf(int v){
    Node* t = ndptr[v];

    vector walk;
    while(t){
        walk.push_back(t);
        t = t->p;
    }
    reverse(walk.begin(), walk.end());
    for(Node* nd : walk){
        push(nd);
        pull(nd);
    }

    t = ndptr[v];
    int idx = sz(t->l);
    while(t){
        if(t->p && t == t->p->r)
            idx += sz(t->p->l) + 1;
        t = t->p;
    }
    return idx;
}

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++)
        scanf("%d", &X[i]);
}

```

```

root = build(0, N);

printf("%d\n", N);
for(int l = 1; l <= N; l++){
    int r = orderOf(l)+1;
    reverse(l, r-1+1);
    printf("%d %d\n", l, r);
}
}

```

Book Shop II

```

#include

using namespace std;
typedef long long ll;
const int maxN = 100, maxK = 1000, maxX = 1e5+1;

int N, X, h[maxN], s[maxN], k[maxN];
ll dp[maxX];

int main(){
    scanf("%d %d", &N, &X);
    for(int i = 0; i < N; i++) scanf("%d", &h[i]);
    for(int i = 0; i < N; i++) scanf("%d", &s[i]);
    for(int i = 0; i < N; i++) scanf("%d", &k[i]);

    fill(dp+1, dp+X+1, -1);
    for(int i = 0; i < N; i++){
        for(int b = 1; k[i] > 0; b++){
            int amnt = min(b, k[i]);
            k[i] -= b;
            int price = amnt * h[i];
            int pages = amnt * s[i];
            for(int j = X; j >= price; j--)
                if(dp[j-price] != -1)
                    dp[j] = max(dp[j], dp[j-price] + pages);
        }
    }

    for(int i = 1; i <= X; i++)
        dp[i] = max(dp[i], dp[i-1]);
    printf("%lld\n", dp[X]);
}

```

Mex Grid Queries

```

#include

using namespace std;

int x, y;

```

```
int main(){
    scanf("%d %d", &x, &y);
    printf("%d\n", (x-1)^(y-1));
}
```

Stick Divisions

```
#include

using namespace std;
typedef long long ll;

int N, x, d;
ll sum;
priority_queue, greater> sticks;

int main(){
    scanf("%d %d", &x, &N);
    for(int i = 0; i < N; i++){
        scanf("%d", &d);
        sticks.push(d);
    }
    while(sticks.size() > 1){
        int a = sticks.top(); sticks.pop();
        int b = sticks.top(); sticks.pop();
        sticks.push(a+b);
        sum += a + b;
    }
    printf("%lld\n", sum);
}
```

Coding Company

```
#include

using namespace std;
typedef long long ll;
const int maxN = 101, maxX = 5e3+1;
const ll MOD = 1e9+7;

int N, X, t[maxN];
ll dp[maxN][maxN][maxX];

int main(){
    scanf("%d %d", &N, &X);
    for(int i = 1; i <= N; i++)
        scanf("%d", &t[i]);
    sort(t+1, t+N+1);
    t[0] = t[1];

    dp[0][0][0] = 1;
```

```
for(int i = 1; i <= N; i++){
    for(int j = N; j >= 0; j--){
        for(int k = X; k >= 0; k--){
            ll cnt = dp[i-1][j][k];
            int newk = k + j * (t[i]-t[i-1]);
            if(newk > X) continue;

            dp[i][j][newk] = (dp[i][j][newk] + (j+1) * cnt) % MOD;
            if(j != N) dp[i][j+1][newk] = (dp[i][j+1][newk] + cnt) % MOD;
            if(j != 0) dp[i][j-1][newk] = (dp[i][j-1][newk] + j * cnt) % MOD;
        }
    }
}

ll tot = 0;
for(int i = 0; i <= X; i++)
    tot = (tot + dp[N][0][i]) % MOD;
printf("%lld\n", tot);
}
```
